Machine learning text classifiers and open and closed-source large language models (LLMs) in political science

Day II: The LLM revolution

Joshua Cova and Luuk Schmitz

2026-04-24

Overview

Part 1: How do LLMs work?

Part 2: Interacting with LLMs in a research setting

Part 3: Open vs. closed-source alternatives

☕ Break

Part 4: Other use cases: simulations, interviews, metadata extraction, agentic augmentation

Part 5: Quantization and locally hosted alternatives

Group Exercise: Local LLMs for text classification

Brainstorming session: How does this apply to your own research?

Part 1: How do LLMs work?

Connecting to yesterday

Yesterday we saw text as a **geometric map**, and meaning as **distance**.

Today, we learn how to **pilot a spaceship** to traverse that geometric and semantic space.

Where we left off

BERT (yesterday): *bidirectional* attention, every word sees every other word.

- Great for **understanding**: classification, NER, scoring text
- Pre-trained by predicting masked tokens
- Fixed output: a representation

Today: a different architecture for a different goal — **generating** text.

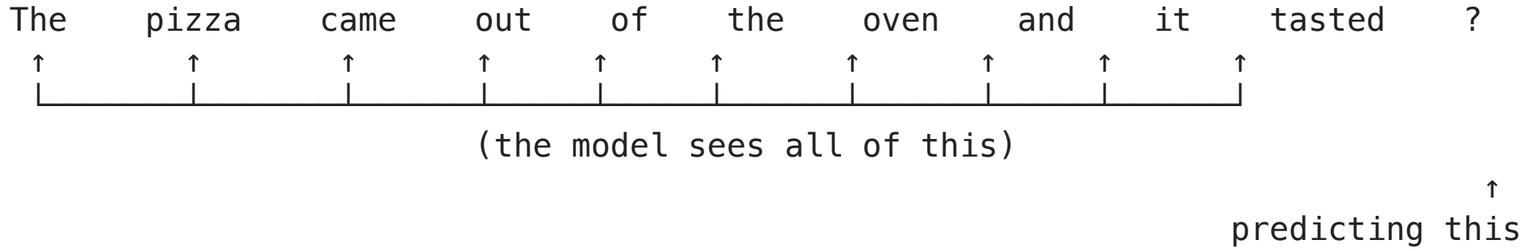
The architectural pivot: encoder → decoder

	Encoder-only (BERT)	Decoder-only (GPT, Claude, Llama)
Attention	Bidirectional	Causal (backward only)
Objective	Masked language modelling	Next-token prediction
Produces	Representations	Text
Good for	Classification, NER, similarity	Generation, chat, reasoning

One architectural change. Enormous behavioural consequences.

Causal attention

Every token attends only to **itself and the tokens before it**.



This single change is what makes **generation** possible: the model can be called repeatedly, each time extending its own output.

Autoregressive generation: one step at a time

At each step:

1. Feed in the input-so-far
2. Model outputs a **probability distribution** over the next token
3. **Sample** one token from the distribution
4. Append it, repeat

Generation is a **trajectory** through the vocabulary space — each token a coordinate, each step a decision.

The training objective

One deceptively simple task:

> Given a sequence, predict the next token.

Repeat across trillions of tokens of text from the web, books, code, papers.

That’s it. That’s the entire pretraining objective.

Why is this so powerful?

To reliably predict the next token in *any* text, the model must implicitly learn:

- Syntax and grammar
- Word meaning in context
- Factual knowledge about the world
- Reasoning patterns
- Stylistic register
- Even aspects of mental states (“Alice believes that Bob…”)

Rich behaviour emerges from a single objective at sufficient scale.

Scale changes everything

Parameters over time:

- GPT-2 (2019): 1.5B
- GPT-3 (2020): 175B
- GPT-4 (2023): ~1.8T (rumored)
- Today’s frontier: trillions

Training data:

- Web text, books, code
- Now: synthetic data, curricula

Capabilities that emerged, mostly unplanned:

- In-context learning
- Chain-of-thought reasoning
- Instruction following
- Tool use
- Code generation

These were not explicitly trained for. They appeared as models got bigger.

Base model vs. assistant

A **pretrained model** is just a text continuation engine.

If you prompt it with “How do I make a soufflé?” it might continue with “This is a question commonly asked by…” — because that’s a plausible continuation on the web.

Post-training turns the base model into an assistant.

The model you chat with (ChatGPT, Claude, Gemini) is **not** GPT or the raw Llama weights. It’s a heavily post-trained artefact.

Example: https://huggingface.co/models?pipeline_tag=text-generation&sort=trending

Post-training: how base models become assistants

Three techniques, usually stacked:

1. **Supervised fine-tuning (SFT)**: show the model many examples of “good” responses to prompts
2. **Reinforcement learning from human feedback (RLHF)**: humans rank outputs; model learns to produce higher-ranked ones (Ouyang et al. 2022)
3. **Direct Preference Optimization (DPO)** and newer variants: simplified versions of the above

Net effect: the model is shaped to be helpful, to follow instructions, to refuse certain things, to format nicely, to avoid offense.

Decoding: how we sample the next token

The model gives a probability distribution. We choose how to sample from it:

- **Temperature**: flattens or sharpens the distribution
  - T = 0: always pick the most likely token (greedy)
  - T = 1: sample from the distribution as-is
  - T = 2: very random
- **Top-p (nucleus) sampling**: sample only from the top tokens that together cover p% of probability mass
- **Seed**: reproducibility knob (where supported)

Temperature in action

Prompt: *“The capital of France is”*

T = 0 (greedy)

```
Paris.
```

T = 0.7

```
Paris, which is the largest city
in the country.
```

T = 1.5

```
Paris, the stunning metropolis
bathed in Seine light.
```

Rule of thumb for research:

- Classification / annotation: **T = 0**
- Open-ended extraction: T = 0 or very low
- Creative generation: T = 0.7–1.0
- Exploration / brainstorming: T = 1.0+

Record the temperature!

Summary

The decoder-only LLM:

- Reads left to right (causal attention)
- Predicts the next token at each step (autoregressive)
- Was trained on trillions of tokens with one objective
- Was then post-trained to be a helpful assistant
- Samples from a distribution, controlled by **temperature**, **top-p**, **seed**

We can now launch. The question is: **where to steer?**

Part 2: Interacting with LLMs in a research setting

The scientific posture

An LLM is a **measurement instrument**, not an oracle.

All the usual concerns apply:

- Construct validity
- Reliability (across runs, across models)
- Inter-rater agreement (with humans, between models)
- Reproducibility

An LLM prompt is an **operationalization**. A change to the prompt mid-project is a change to your coding scheme.

The empirical promise

Since 2023, study after study has shown LLMs matching or beating expert coders on social-science classification tasks.

- Gilardi, Alizadeh, and Kubli (2023): zero-shot ChatGPT beats crowd workers on stance, relevance, topic, frame detection
- Törnberg (2023): GPT-4 beats expert coders *and* fine-tuned BERT at classifying politicians' tweets across 11 countries
- Heseltine and Clemm Von Hohenberg (2024): GPT-4 substitutes for human experts across annotation tasks in political science
- Le Mens and Gallego (2025): LLMs scale political texts along left-right dimensions at human-coder accuracy
- Mellon et al. (2024): LLMs code open-text survey responses with accuracy similar to human annotators

The cost collapse

Le Mens and Gallego (2025) scale 900 tweets on left-right ideology.

	Human coding	**LLM (GPT-4)**
Cost	£1,626 (crowdsourced)	**$1.50**
Speed	Days	Minutes
Reproducibility	Low (different coders)	High (fixed model + prompt)
Accuracy	Baseline	Matches or beats baseline

Running Llama 3 locally? **Free.** Slower, but free.

But — caveats appear quickly

Before we get too excited, the same literature has documented serious limitations.

- Performance is **highly task-dependent**: works great on some, fails on others
- Performance is **highly prompt-dependent**: rephrasings change accuracy by 10–20 points (Atreja et al. 2025)
- Performance is **highly model-dependent**: GPT-4 ≠ Llama ≠ Claude
- Performance varies across **languages and domains**
- Models **hallucinate**, **flatter users**, **over-rely on cues**, **memorize training data**

Individual validation is non-negotiable.

Prompt engineering is instrument design

Three basic prompting strategies:

Zero-shot

Give instructions, no examples.

```
Classify this tweet as
Democrat or Republican.

Tweet: "..."

Label:
```

Few-shot

Give instructions + a handful of labeled examples.

```
Examples:
Tweet X → Democrat
Tweet Y → Republican
Tweet Z → Democrat

Tweet: "..."
Label:
```

Chain-of-thought

Ask the model to reason before answering.

```
Classify this tweet.
Reason step by step
about policy positions,
then give your label.
```

The codebook-construct label assumption

Halterman and Keith (2026) argue: forget the "universal label assumption" where we just pass the LLM a label name and trust pretraining.

Instead, make the **codebook-construct label assumption**: the LLM should follow the definition and exclusion criteria supplied in a codebook.

Lazy (universal)	**Disciplined (codebook-based)**
"Label as *protest* or *riot*"	"Definition of *protest*: … Definition of *riot*: … Positive examples: … Negative examples: … Exclusion criteria: …"

10–20 percentage points of accuracy live in this difference.

A validation protocol in three levels

1. **Syntactic correctness**: does the LLM produce one of your valid labels?
2. **Robustness**: do predictions survive innocuous changes (label order, rephrasing, exemplar shuffling)?
3. **Human gold standard**: agreement on a hand-coded sample (200–500 docs)

Level 1: Syntactic correctness

The simplest test: does the LLM produce one of the valid labels?

```
Codebook labels: {DEMOCRAT, REPUBLICAN, OTHER}

LLM outputs:
✓ "DEMOCRAT"
✓ "OTHER"
✗ "The tweet appears to lean Republican based on..."
✗ "I'm not sure, but probably Democrat"
```

Even at T=0.0, over long classification runs, a surprising number of model-prompt combinations fail this basic test.

Halterman and Keith (2026) test this as “Test I” — checking if any user-defined label appears in the output.

Level 2: robustness tests

Good predictions should not change when you do things that shouldn't matter.

- **Label order**: shuffle the order of labels in the prompt. Do predictions change?
- **Exemplar order** (few-shot): shuffle exemplars. Do predictions change?
- **Paraphrasing**: rewrite the instructions. Does accuracy hold?
- **Seed**: re-run at T=0 with a different seed. How stable are outputs?

Halterman and Keith (2026)'s "Test IV": Fleiss' κ across original, reversed, and shuffled codebooks. If κ is low, the model isn't really comprehending your codebook — it's pattern-matching on surface features. See also: Benoit et al. (2026).

Level 3: human gold standard

The traditional content-analysis validation, unchanged.

1. Hand-code a sample (200–500 documents) with at least two trained coders
2. Compute inter-coder reliability (κ, α)
3. Run your LLM pipeline on the same sample
4. Compute LLM-human agreement (accuracy, F1, κ)
5. Report **all of it** in the paper

If LLM-human agreement ≈ human-human agreement, the LLM is a valid coder.

Known failure modes

LLMs fail not only in familiar ways, but also in ways that require a new epistemic framework to even think about (Törnberg 2024).

1. **Sycophancy**: agreeing with the user's stated views
2. **Political cue bias**: over-reliance on partisan signals
3. **Data contamination**: memorizing rather than inferring
4. **Artificial precision**: exaggerating differences between groups

Each is documented. Each has mitigation strategies. **All of them require validation to detect.**

Failure mode 1: sycophancy

LLMs have a tendency to tell users what they want to hear.

Sharma et al. (2023) showed that simply rephrasing a question to include the user's stated opinion shifts the model's answer toward that opinion — even on objective questions.

For research:

- If your prompt embeds an expectation ("*Do you agree this tweet is hateful?*"), the model tilts toward agreement
- If you describe yourself in the system prompt, the model adapts
- **Use neutral framing**: "*Classify this tweet as hateful or not.*"

Failure mode 2: political cue bias

A 2025 study in *Humanities and Social Sciences Communications* found:

> When annotating political statements, LLMs are **more** sensitive to partisan cues than human coders (Vallejo Vera and Driggers 2025).

Same statement, attributed to different parties → different LLM labels.

For polarization research, this is a validity threat. If your outcome depends on LLM-coded text, and the LLM is over-weighting partisan signals the human coders do not, you will infer more polarization than actually exists.

Failure mode 3: data contamination

The problem: LLMs were trained on the internet. The internet includes most of the political texts you might want to analyze.

Deng et al. (2023) showed GPT-4 can guess masked options in standard benchmarks with 57% accuracy — strong evidence of memorization.

When you ask an LLM to classify a Reagan speech, is it inferring, or retrieving?

Failure mode 4: artificial precision

Bisbee et al. (2024) documented a striking pattern in “silicon samples”:

ChatGPT, when prompted to simulate political groups’ opinions, produces responses with **artificially low variance**.

Groups look more homogeneous than they are. Differences between groups look sharper than they are.

For any study inferring population heterogeneity from LLM output: beware. The model has learned stereotypes, not distributions.

Takeaways for Part 2

1. An LLM is a **measurement instrument**, not an oracle
2. The prompt is your **operationalization** — it is the codebook
3. Validate at four levels: legal output, robustness, human gold standard, surrogate inference
4. Know the failure modes: sycophancy, political cue bias, contamination, artificial precision
5. Report everything: model, version, prompt, temperature, date

Recommended primers: Törnberg (2024), Benoit et al. (2026); Halterman and Keith (2026); Cova and Schmitz (2024)

Part 3: Open vs. closed-source alternatives

Not a binary — a spectrum

Fully closed

GPT, Claude, Gemini

- API access only
- Weights secret
- Training data undisclosed
- State-of-the-art

Open weights

Llama, Mistral, Qwen, DeepSeek, Gemma

- Download and run
- Training data mostly undisclosed
- Permissive licenses (mostly)

Fully open

OLMo, Pythia

- Weights + data + code
- Full reproducibility
- Smaller / older
- Research-grade

The tradeoff matrix

	Closed API	Open weights (local)
Capability	Frontier	Very strong, catching up
Setup cost	Near zero	Real (hardware, serving)
Marginal cost	Per token	Zero (after electricity)
Data privacy	Data leaves your infra	Stays on your machine
Reproducibility	Silent drift	Frozen weights
Rate limits	Real	None
Capability ceiling	Highest	Lower but narrowing

The capability gap is narrowing — for most tasks

For state-of-the-art classification tasks (political text, sentiment, stance), the gap between GPT-5 and a fine-tuned Llama 3 70B is often **negligible**.

Alizadeh et al. (2025) benchmark fine-tuned open-source LLMs on political science annotation tasks: they match or exceed zero-shot GPT-4.

Where the gap persists:

- Long-context reasoning (100k+ tokens)
- Agentic multi-step tasks
- Some non-English languages
- Niche domains

Why this matters for social science specifically

Four domain-specific considerations:

1. **Data ethics**: interview transcripts, archival material with PII. Closed APIs send this data to third-party servers
2. **Reproducibility**: closed models drift silently. For a paper meant to be replicable for decades, open weights are safer
3. **Cost at scale**: 500k documents × prompt tokens = real money. Non-trivial for doctoral budgets
4. **Norms are shifting**: reviewers ask for model version, prompt, temperature. Soon they'll ask for open weights for reproducibility.

A decision heuristic

```
IF data is sensitive (interviews, PII, confidential)
    → open weights, local
ELIF volume is high (>100k documents)
    → open weights, local (cost)
ELIF you need frontier reasoning / long context / agents
    → closed API
ELSE
    → start with whichever is easier; validate; switch if needed
```

For most classification work: open weights locally. **For exploration and frontier tasks**: closed API. **For published reproducible work**: open weights, version-pinned.

Publication norms: still emerging

In political science and adjacent disciplines, the standards and expectations for transparency and reproducibility of LLM-guided research are still in their infancy - lagging behind other disciplines.

Nevertheless, you should (arguably) **always** be able to answer the following questions:

- “Which model did you use? What hyperparameters? What prompting strategy?
- Pre-registration of LLM protocols?
- LLM artifacts (prompts, model info, test/train data) as part of replication packages
- How reproducible is reproducible enough? –> Unresolved question

The frontier shifts fast. The norms are catching up.

Break (15 min)

Part 4: Other use cases

Beyond classification

So far we've focused on **text classification**. LLMs do much more:

1. **Simulations**: using LLMs as synthetic survey respondents
2. **Interviews**: coding qualitative data, or being the respondent
3. **Metadata extraction**: unstructured text → structured fields
4. **Agentic augmentation**: LLMs as research collaborators that *execute*

We'll spend most of our time on **metadata extraction** (immediate wins) and **agentic augmentation** (where things are going).

Simulations: the promise

Argyle et al. (2023) (*Political Analysis*, “Out of one, many”):

> Condition GPT-3 on a demographic profile, ask political questions, get responses that correlate with real ANES data.

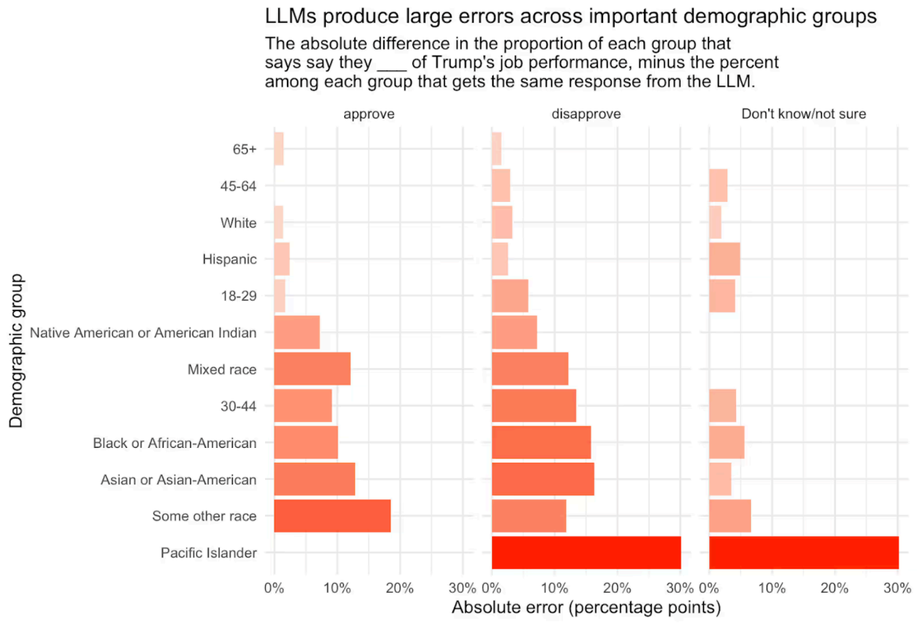
Correlations above 0.9 on voting preferences.

The dream: *“silicon samples”* — LLM-generated synthetic respondents for piloting instruments, augmenting small samples, running counterfactual experiments.

Simulations: the reality check

Morris (2025) (Verasight, three whitepapers):

- At the **topline** (overall population %): accuracy within 4 points of real surveys ✓
- At the **subgroup level**: error averaging 10 points, up to **30 points** for small subgroups ✗
- **Market research** (brand awareness, product testing): much worse than politics — training data has more on elections than on coffee



Simulations: the reality check

- **Structural inconsistency**: proportional accuracy fails across demographic aggregations
- **Homogenization**: minority opinions within simulated subgroups are underrepresented

Simulations: the verdict

LLMs as silicon samples are a **generative hypothesis engine**, not a substitute for human respondents.

Legitimate uses:

- Pilot studies: is this question going to work?
- Counterfactuals: what would happen if we asked it differently?
- Small-sample augmentation (with heavy validation)
- Stress-testing instruments before fielding

Illegitimate uses:

- Reporting population-level findings
- Subgroup analysis without real data
- Anything where you'd cite a p-value

Interviews: two modes

LLM as coder of transcripts

- Thematic analysis
- Coding frames, stance, sentiment
- Mellon et al. (2024): open-text survey responses at human accuracy

Same methodology as Part 2 — validate, validate, validate.

LLM as synthetic interviewer / respondent

- More speculative
- Shows real promise: Chopra and Haaland (2023)
- Loses the ethnographic encounter
- But: can scale, standardize, and people seem to (slightly) prefer it

A note from the qualitative side

A critique worth taking seriously:

> AI-assisted qualitative analysis risks a drift back toward positivism — treating meaning as something to be **extracted** from data rather than **constructed** through the researcher's situated engagement with it.

— Lozano Paredes (response to Tabor 2026)

The methodological question: when the AI identifies "themes" across interview transcripts, is that interpretation, or pattern recognition disguised as interpretation?

Metadata extraction: Potentially very useful

Take unstructured text → produce **structured, typed data**.

- Legislation → actor, action, date, affected population
- News articles → event coding (LLM-era successor to ACLED, CAMEO)
- Corporate filings / earnings calls → structured claims
- Interview transcripts → quote + theme + speaker metadata

LLMs now support structured outputs (output that adheres to a pre-defined schema). This allows us to turn unstructured text into structured data (JSON, CSV) at scale.

See: https://simonwillison.net/2025/Feb/28/llm-schemas/

Structured extraction in practice

```python
from openai import OpenAI
from pydantic import BaseModel

client = OpenAI()

class CalendarEvent(BaseModel):
    company: str
    date: str
    amount: int
    country: str

response = client.responses.parse(
    model="gpt-4o-2024-08-06",
    input=[
        {"role": "system", "content": "Extract the event information."},
        {
            "role": "user",
            "content": "Apple will invest US$3bn in a new assembly line in Thailand in 2026.",
```

Agentic augmentation

We’ve been discussing LLMs as tools that **talk**.

A new class of systems treats LLMs as collaborators that **execute**.

Give the model:

- A goal
- Access to tools (file system, code execution, web search, APIs)
- A bit of context

The model will **plan, act, observe, and iterate**.

What has happened

A few data points from late 2025 / early 2026:

- **METR**: AI task duration doubling roughly every seven months — from 1-hour tasks in early 2025 to 8-hour workstreams by late 2026
- **Karpathy**: shifted from ~80% manual to ~80% agent coding in weeks
- People are producing publishable research papers in a matter of days, if not hours. See https://statsandsociety.substack.com/p/you-should-absolutely-be-freaking
- **APE (Zurich)**: 200+ economics papers generated autonomously

What an agentic research setup looks like

- Works in VS Code, with Claude Code running inside
- Entire research workflow in one environment: Python, LaTeX, references, writing, websites
- Hierarchy of `CLAUDE.md` files: one for the research program, one per domain, one per project
- The AI *reads your files*, *understands your argument*, *accumulates context across sessions*
- It doesn't just answer questions — it plans, executes, verifies

The metaphor: **"a room full of experts while you think"**.

https://statsandsociety.substack.com/p/you-should-absolutely-be-freaking

A concrete example: agentic peer review

Claes Bäckman's `review-paper` skill for Claude Code.

Six specialized sub-agents run in parallel on your draft:

1. Spelling, grammar, academic style
2. Internal consistency, cross-reference verification
3. Unsupported claims, identification integrity
4. Mathematics, equations, notation
5. Tables, figures, documentation
6. Journal-specific adversarial referee (QJE, AER, JF, …)

Consolidated pre-submission report. **~100 lines of markdown. MIT-licensed. On GitHub.**

Source: https://github.com/claesbackman/AI-research-feedback

The caution: agency amplifies errors

Tabor's anecdote:

> Claude Code proposed an elaborate extension of a theoretical claim, with a formal proposition and a full proof.
>
> Technically impressive, logically tight, even a bit elegant.
>
> But the proof assumed a *probability distribution* where our paper explicitly specifies the parameters as *deterministic*.
>
> Plausible. Well-formatted. Substantively wrong.

The methodological discipline

The pattern that works:

1. **Discuss before execute.** Present your ideas. Let the AI push back. Converge on a plan.
2. **Use Planning Mode**: AI presents structured plan, you review and approve.
3. **Execute incrementally**, with human verification of each step.
4. **Check everything.** Recreate results independently where possible.
5. **Domain expertise is the early-warning system.** Cultivate it.

The skill is no longer in the doing. The skill is in the **directing** and the **verifying**.

A provocation to end on

Some institutional consequences to agentic AI research:

- Manuscript creation: 2–3 hours, ~$100
- Submissions could increase **5x** while journal slots stay fixed
- Desk rejection: from ~50% to ~90%
- Peer review: unsustainable at current scale

If a generalist can produce a Q1-quality paper with a day of prompting —

what's the distinctive contribution you'll make as a researcher three years from now?

See: https://www.popularbydesign.org/p/academics-need-to-wake-up-on-ai

Part 5: Quantization and locally hosted alternatives

Why run LLMs locally?

- **Data privacy**: sensitive texts never leave your machine
- **Reproducibility**: weights are frozen files, not a moving API
- **Cost**: zero marginal cost after hardware
- **Independence**: no rate limits, no vendor drift
- **Tinkering**: fine-tune, inspect, debug

The obstacle: **memory**.

The memory problem

A model's memory footprint ≈ parameters × bytes per parameter.

Llama 3.1 70B at full precision (FP16):

$$70 \text{ billion} \times 2 \text{ bytes} = 140 \text{ GB}$$

No laptop has 140 GB of VRAM. Even many university GPUs don't.

Solution: quantization.

Quantization: compressing the model

Store weights at lower precision.

Precision	Bytes per param	70B model
FP32 (original training)	4	280 GB
FP16 (standard inference)	2	140 GB
Q8 (8-bit)	1	70 GB
Q4 (4-bit)	0.5	**35 GB**
Q2 (2-bit)	0.25	~17 GB (quality hit)

Q4 is the sweet spot: ~4x smaller, minimal quality loss on most tasks.

The hardware ladder

What you can realistically run locally:

Hardware	VRAM/RAM	What fits at Q4
Laptop (16 GB RAM)	16 GB	7–8B models comfortably
Apple Silicon (32–64 GB)	32–64 GB	30–70B models
RTX 4090 / 5090	24–32 GB	13–32B models
University GPU node (A100/H100)	40–80 GB	Whatever you want
Multi-GPU cluster	∞	Frontier-scale

For **most classification tasks**: a 7–8B model is plenty.

The ecosystem, ranked by user-friendliness

1. **Ollama**: one command to run a model. “Docker for LLMs.” Start here.
2. **LM Studio**: GUI on top of the same ideas. Good for non-command-line users.
3. **llama.cpp / GGUF**: under the hood of both. For when you want to tune.
4. **Hugging Face Transformers**: Python-native, flexible, higher compute needs.
5. **vLLM / TGI**: production serving. When you need throughput.

For the afternoon exercise: we’ll use **Ollama**.

What to expect

Running Llama 3.1 8B locally (Q4) on a modern laptop:

- **Speed**: 20–60 tokens/second (usable for moderate batch work)
- **Quality on classification**: typically within 2–5 points of GPT-4 on well-defined tasks
- **Cost**: zero (once you've bought the laptop)
- **Latency**: low, no network roundtrip
- **Reliability**: fully local, no rate limits

For a 10,000-document classification task: an afternoon, not a weekend. And no API bill.

Transitioning to the exercise

You now know:

- How LLMs work (Part 1)
- How to use them as research instruments (Part 2)
- Open vs. closed tradeoffs (Part 3)
- The broader use-case landscape (Part 4)
- How to run them locally (Part 5)

This afternoon:

- Install Ollama
- Classify a text corpus locally
- Compare against a human-coded subset
- Iterate on the prompt
- Bring the questions to the brainstorming session

Let’s eat lunch, then build something.

References

Alizadeh, Meysam, Maël Kubli, Zeynab Samei, Shirin Dehghani, Mohammadmasiha Zahedivafa, Juan D. Bermeo, Maria Korobeynikova, and Fabrizio Gilardi. 2025. “Open-Source LLMs for Text Annotation: A Practical Guide for Model Setting and Fine-Tuning.” *Journal of Computational Social Science* 8 (1): 17. https://doi.org/10.1007/s42001-024-00345-9.

Argyle, Lisa P., Ethan C. Busby, Nancy Fulda, Joshua R. Gubler, Christopher Rytting, and David Wingate. 2023. “Out of One, Many: Using Language Models to Simulate Human Samples.” *Political Analysis* 31 (3): 337–51. https://doi.org/10.1017/pan.2023.2.

Atreja, Shubham, Joshua Ashkinaze, Lingyao Li, Julia Mendelsohn, and Libby Hemphill. 2025. “What’s in a Prompt?: A Large-Scale Experiment to Assess the Impact of Prompt Design on the Compliance and Accuracy of LLM-Generated Text Annotations.” *Proceedings of the International AAAI Conference on Web and Social Media* 19 (June): 122–45. https://doi.org/10.1609/icwsm.v19i1.35807.

Benoit, Kenneth, Scott De Marchi, Conor Laver, Michael Laver, and Jinshuai Ma. 2026. “Using Large Language Models to Analyze Political Texts Through Natural Language Understanding.” *American Journal of Political Science*, March, ajps.70050. https://doi.org/10.1111/ajps.70050.

Bisbee, James, Joshua D. Clinton, Cassy Dorff, Brenton Kenkel, and Jennifer M. Larson. 2024. “Synthetic Replacements for Human Survey Data? The Perils of Large Language Models.” *Political Analysis* 32 (4): 401–16. https://doi.org/10.1017/pan.2024.5.

Chopra, Felix, and Ingar Haaland. 2023. “Conducting Qualitative Interviews with AI.” *SSRN Electronic Journal*. https://doi.org/10.2139/ssrn.4583756.

Cova, Joshua, and Luuk Schmitz. 2024. “A Primer for the Use of Classifier and Generative Large Language Models in Social Science Research.” https://doi.org/10.31219/osf.io/r3qng.

Deng, Chunyuan, Yilun Zhao, Xiangru Tang, Mark Gerstein, and Arman Cohan. 2023. “Investigating Data Contamination in Modern Benchmarks for Large Language Models.” arXiv. https://doi.org/10.48550/ARXIV.2311.09783.

Gilardi, Fabrizio, Meysam Alizadeh, and Maël Kubli. 2023. “ChatGPT Outperforms Crowd Workers for Text-Annotation Tasks.” *Proceedings of the National Academy of Sciences* 120 (30): e2305016120. https://doi.org/10.1073/pnas.2305016120.

Halterman, Andrew, and Katherine A. Keith. 2026. “Codebook LLMs: Evaluating LLMs as Measurement Tools for Political Science Concepts.” *Political Analysis* 34 (2): 188–204. https://doi.org/10.1017/pan.2025.10017.

Heseltine, Michael, and Bernhard Clemm Von Hohenberg. 2024. “Large Language Models as a Substitute for Human Experts in Annotating Political Text.” *Research & Politics* 11 (1): 20531680241236239. https://doi.org/10.1177/20531680241236239.

Le Mens, Gaël, and Aina Gallego. 2025. “Positioning Political Texts with Large Language Models by Asking and Averaging.” *Political Analysis* 33 (3): 274–82. https://doi.org/10.1017/pan.2024.29.

Mellon, Jonathan, Jack Bailey, Ralph Scott, James Breckwoldt, Marta Miori, and Phillip Schmedeman. 2024. “Do AIs Know What the Most Important Issue Is? Using Language Models to Code Open-Text Social Survey Responses at Scale.” *Research & Politics* 11 (1): 20531680241231468. https://doi.org/10.1177/20531680241231468.

Morris, G. Elliott. 2025. “Your Polls On ChatGPT A Report Discussing the Challenges of ‘Synthetic Sampling’ for Public Opinion Polling.” *Verasight*. https://www.verasight.io/reports/synthetic-sampling.

Ouyang, Long, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, et al. 2022. “Training Language Models to Follow Instructions with Human Feedback.” arXiv. https://doi.org/10.48550/ARXIV.2203.02155.

Sharma, Mrinank, Meg Tong, Tomasz Korbak, David Duvenaud, Amanda Askell, Samuel R. Bowman, Newton Cheng, et al. 2023. “Towards Understanding Sycophancy in Language Models.” arXiv. https://doi.org/10.48550/ARXIV.2310.13548.

Törnberg, Petter. 2023. “ChatGPT-4 Outperforms Experts and Crowd Workers in Annotating Political Twitter Messages with Zero-Shot Learning.” https://doi.org/10.48550/ARXIV.2304.06588.

———. 2024. “Best Practices for Text Annotation with Large Language Models.” https://doi.org/10.48550/ARXIV.2402.05129.

Vallejo Vera, Sebastián, and Hunter Driggers. 2025. “LLMs as Annotators: The Effect of Party Cues on Labelling Decisions by Large Language Models.” *Humanities and Social Sciences Communications* 12 (1): 1530. https://doi.org/10.1057/s41599-025-05834-4.

Wei, Jason, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. 2022. “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models.” arXiv. https://doi.org/10.48550/ARXIV.2201.11903.